Assignment 2 – CO2

Functions, Classes, Static Members, Friend and Inline Functions

Field	Details
Name / Register No.	Kaviya V/192421022
GitHub repository	https://github.com/kaviyavellaisamy/DSA0107---C-/tree/main

Predictions are hand-traced before running; actual outputs, warnings, error messages and timings were produced by compiling and running the code (g++ -std=c++17 -Wall -Wextra). Timings come from a single machine and will differ on yours – attach your own.

Q1 – Call by Value, by Reference and by Pointer (6 marks)

Three versions of the concession function

```
void applyByValue(long long fee, int percent)    { fee  -= fee  * percent / 100; }  // local copy
void applyByRef  (long long& fee, int percent)   { fee  -= fee  * percent / 100; }  // alias of caller's fee
void applyByPtr  (long long* fee, int percent)   { *fee -= *fee * percent / 100; }  // address of caller's fee
```

Call (each starts from fee = 45000, 10 %)	Predicted stored fee	Actual (run)
applyByValue(fee, 10)	45000 (unchanged: only the copy changes)	45000
applyByRef(fee, 10)	40500	40500
applyByPtr(&fee, 10)	40500	40500
Returning version, chained: applyReturn(applyReturn(fee, 10), 15)	45000 → 40500 → 34425	34425

Inquiry

- By value: the parameter is a new local variable on the callee's stack frame, initialised with a copy. Changing it cannot affect the caller – the printed value inside the function looks right, the stored fee never changes.
- By reference: the parameter is another name for the caller's object; no copy; changes are visible to the caller.
- By pointer: the parameter is a local copy of the address; dereferencing it reaches the caller's object. Only the pointee changes, so two of three versions change the caller's fee.
- Use **const reference** for large objects that are only read (const Student&): no copy and no modification possible.

Design decision

Several concessions may apply to one student, so return the new fee (long long applyConcession(long long fee, int percent)) and let the caller combine: fee = applyConcession(applyConcession(fee, 10), 15);.

A pure function is easier to test and chain and cannot half-update a student on an error. In-place change is reasonable when the object is large and updated frequently. A decision is also needed on the order: percentages are applied one after another (not added), and integer division rounds down each time.

Predict the Output – prediction and actual

```
1 12 13
1 100
16 13 6
```

Line	Explanation
1 12 13	change(a, b, &c): a is by value (stays 1); b is a reference (2 → 12); c is a pointer target (3 → 13).
1 100	bigger(a, c) returns a reference to the larger of 1 and 13, i.e. c, and the call on the left of = assigns 100 to c. a stays 1.
16 13 6	show(1) uses both defaults: 1+5+10 = 16. show(1,2) uses default z: 1+2+10 = 13. show(1,2,3) uses none: 6.

Complete the Code – TODO 1

```
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;     // integer arithmetic: 45000 - 4500 = 40500
}
```

```
Actual output:
after call by value : 45000
after call by reference: 40500
```

Call by value leaves the fee at 45000 because applyByValue reduced only its own copy, which was destroyed on return.

Alternative concept – return the new fee

Time is O(1) in all versions. Space copied per call: by value copies sizeof(long long) = 8 bytes (negligible); for a large Student object (say 200 bytes) by value copies all 200 bytes on every call, by reference and by const reference copy only an address (8 bytes). Returning the fee by value copies 8 bytes back. For primitive types by-value is fastest; for large objects use const reference.

Q2 – Return by Reference and Its Dangers (4 marks)

```
int& highest(int marks[], int n) {              // reference into the caller's array
    int best = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[best]) best = i;
    return marks[best];
}
int& bad() { int x = 5; return x; }             // DANGER: x dies when bad() returns

int main() {
    int m[] = {55, 78, 62};
    highest(m, 3) += 2;                         // moderation applied directly -> m[1] = 80
    highest(m, 3) = 90;                         // call used on the left of =
}
```

Return type	What the caller can do	Safety
int& (reference)	Read and modify the element; call may appear on the left of =	Safe only while the object lives
const int&	Read without copying; cannot modify	Safe while the object lives
int (copy)	Read only a copy; modifying it changes nothing	Always safe; copies the value

Compiler warning evidence: g++ -Wall prints "warning: reference to local variable 'x' returned [-Wreturn-local-addr]". Using the result is undefined behaviour because the stack slot of x is reused by later calls – the program may print 5, garbage, or crash.

Changed condition – std::vector that grows

A reference (or pointer / iterator) to v[i] points to an address inside the vector's buffer. When push_back exceeds capacity the vector allocates a bigger buffer, moves the elements and frees the old one. Test result: after 1000 push_backs the buffer address changed ("buffer moved: 1"). The earlier reference now dangles, so using it is undefined behaviour. Obtain the reference again after growth, or use an index.

Alternative concept – return an index or a pointer

```
int highestIndex(const int m[], int n);      // caller: m[highestIndex(m,n)] += 2;
int* highestPtr(int m[], int n);              // caller: *highestPtr(m,n) += 2;
```

Time to find the highest mark is O(n) in all three; extra space O(1). Safety when the array becomes a growing vector: an **index stays valid** after growth (v[i] is re-computed from the new buffer); a **pointer and a reference become invalid**. So an index is the most robust return value for growing containers; a reference is the most convenient for fixed arrays.

Q3 – Default Arguments and Function Overloading (6 marks)

The compiler error

With charge(double), charge(double, bool) and charge(double, bool = false, bool = false), the call charge(2.0) matches charge(double) and also the third function (using both defaults). The compiler cannot choose: **error: call of overloaded 'charge(double)' is ambiguous** (reproduced). charge(2.0, true) is ambiguous between the second and third functions in the same way.

Redesign

```
struct Parcel { double kg; };
double charge(int grams,       bool express = false, bool insured = false);
double charge(double kg,       bool express = false, bool insured = false);
double charge(const Parcel& p, bool express = false, bool insured = false);
```

One function per kind of input (int grams, double kg, Parcel), and the optional flags are default arguments. The three overload sets differ in the **first parameter type**, so no call is ambiguous.

Predicted selection for six calls (before running) and observed

Call	Predicted	Observed	Rule
charge(2)	int grams	int-grams	Exact match int
charge(2.0)	double kg	double-kg	Exact match double
charge(2.0f)	double kg	double-kg	float → double is a promotion; float → int is a conversion; promotion wins
charge(500, true)	int grams	int-grams	Exact match + default insured
charge(Parcel{1.5}, true, true)	Parcel	Parcel	Exact match class type
charge(1.2, false, true)	double kg	double-kg	Exact match

Inquiry – how the compiler chooses; why defaults go from the right

Overload resolution ranks each argument conversion: (1) exact match (including trivial const/reference changes), (2) promotion (float → double, char/short/bool → int), (3) standard conversion (int → double, double → int, derived-to-base pointer), (4) user-defined conversion. The function whose conversions are best for every argument wins; if two are equally good the call is ambiguous. Default arguments must be given from the rightmost parameter leftwards because arguments are matched left to right: a call f(1) can only skip trailing parameters, so a default on a left parameter with none to its right could never be used.

Alternative concept – one Parcel structure with optional fields, or a template

Overload resolution happens at compile time, so there is no run-time cost in either design. Space copied per call: a Parcel passed by const reference copies only an address; by value it copies all its fields (maybe 16–24 bytes). A new option (say fragile) with overloads means adding another bool default to every overload and re-checking for ambiguity; with a Parcel it is one new field and no new function. A template (template<class W> double charge(W w, ...)) removes duplicate overloads but accepts types for which no rate exists and gives worse error messages.

Q4 – Specifying a Class with an Array Inside It (6 marks)

```
class SavingsAccount {
    long accNo;
    double balance;
    double txn[10];            // last 10 transactions (+ deposit, - withdrawal)
    int count;                 // 0..10
public:
    SavingsAccount(long no, double bal) : accNo(no), balance(bal), count(0) {}
    void deposit(double amt)  { balance += amt; record(+amt); }       // defined inside
    void withdraw(double amt);                                         // defined outside
    void miniStatement() const;
private:
    void record(double amt) {
        if (count == 10) { for (int i = 1; i < 10; ++i) txn[i-1] = txn[i]; txn[9] = amt; }  //
shift
        else txn[count++] = amt;
    }
};
void SavingsAccount::withdraw(double amt) { if (amt <= balance) { balance -= amt; record(-
amt); } }
void SavingsAccount::miniStatement() const {
    for (int i = 0; i < count; ++i) std::cout << txn[i] << '\n';
```

```
}
```

Inquiry

- **Why private:** if the array were public any code could write txn[3] = 99999 and the statement would no longer match the balance; private data lets record() maintain the invariants (count ≤ 10, order, matching balance).
- **11th transaction – shifting:** move 9 elements down and put the new one at the end: O(k) per add (k = 10). Order is always oldest-first so printing is simple.
- **11th transaction – circular index:** overwrite the oldest slot at (start + count) % 10 and advance start: O(1) per add; printing starts at start and wraps.
- Measured for 10,000,000 adds (-O2): shifting 10 elements 85 ms, circular 39 ms; shifting 50 elements 134 ms (-O0: 1192 ms), circular 38 ms (-O0: 49 ms). The circular version stays flat as k grows.

Design decision – last 50 transactions

Change one constant (const int N = 10 → 50) and the array size; the member function logic is unchanged because it uses N, not a literal 10. With shifting the cost per add grows from 9 to 49 element moves; with a circular buffer it stays O(1). So **the circular buffer changes least in performance** and the constant-based design changes least in code.

Alternative concept – std::deque or circular buffer

std::deque: push_back, and pop_front when size is 10 – O(1) per add; space O(k) with some block overhead, and dynamic allocation. Circular array: O(1) per add, exactly k doubles + an index, no allocation. Shifted array: O(k) per add, k doubles. For a fixed small k the circular array is the most space-efficient and the fastest.

Q5 – Static Members and Arrays of Objects (7 marks)

Design from the principle

- "Data that belongs to the whole class" → **static data members**: issued (count of seats issued) and CAPACITY.
- A static data member is declared inside the class but must be **defined once outside it** (int BusTicket::issued = 0;) – otherwise the linker reports an undefined reference.
- A **static member function** (count()) belongs to the class, has no this pointer, and can therefore touch only static members – it is callable as BusTicket::count() without any object.
- An array of objects BusTicket trip[4] = {BusTicket("Asha"), ...} runs the constructor once per element, in index order.
- The count keeps growing across loops and function calls because it is not stored in any object; objects destroyed later do not reduce it unless a destructor does so.

Predict the Output (Token) – prediction and actual

```
101 103 104 4
```

next starts at 100 (defined as int Token::next = 100 before main). Token counter[3] runs the constructor three times: ids 101, 102, 103. Token extra is the fourth call: 104. issued() = next − 100 = 104 − 100 = 4. counter[0].get() = 101, counter[2].get() = 103, extra.get() = 104. The constructor ran 4 times.

Complete the Code – TODO 1 and TODO 2

```
explicit BusTicket(const std::string& n) : seat(0), name(n)
{
    if (issued < CAPACITY) {            // TODO 1
        ++issued;
        seat = issued;
    }                                    // otherwise seat stays 0 = waiting list
}
static int count()
{
    return issued;                       // TODO 2
}
```

```
Actual output:
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Tickets created in a function called twice: the first call issues seats 1–3 (and Dev waits); the second call finds issued == CAPACITY, so every ticket is on the waiting list. The static count persists between calls, so a second bus would need its own counter – see below.

Alternative concept – global counter or TicketOffice object

Time O(1) and space (one counter) are the same in all three. A global counter can be modified by any code (no encapsulation) and allows only one bus. A TicketOffice object passed to every BusTicket gives one counter per bus (two buses at once) and good encapsulation, but every constructor needs an extra argument. A static member is encapsulated and simple but is one counter for the whole program – no second bus.

Q6 – Friend Functions and Inline Functions (8 marks)

Design

- A **friend function** is a non-member that the class grants access to private data. It is the natural choice when the operation treats two objects **equally** (minutesBetween(a, b)); a member function a.minutesTo(b) makes the left object special.
- Call form: a friend is called minutesBetween(a, b) (no dot, no this); a member is a.minutesTo(b) and has an implicit this. A friend can read private h and m of both arguments; it is not a member and is not inherited.
- **inline** is a request/permission for the compiler to replace the call by the body; it also allows the definition in headers without multiple-definition errors. The compiler may ignore it, and at -O2 it inlines small functions even without the keyword.
- Making h and m public would let any code set h = 99, breaking the invariant $0 \le h < 24$; encapsulation would be lost.

Complete the Code – TODO 1 and TODO 2

```
int minutesBetween(const Time& a, const Time& b)
{
    return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));     // needs <cstdlib>
}
```

```
bool earlier(const Time& a, const Time& b)
{
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
```

```
Actual output:
205 false
```

Check: 9:45 = 585 min, 13:10 = 790 min, difference 205. earlier(arrival, departure): 790 < 585 is false.

Correctness checks

Case	Hand-calculated expectation	Actual (run)
Same time twice (9:45, 9:45)	minutes = 0, earlier = false	minutes=0 earlier=false
11:50 vs 12:10 (either side of noon)	minutes = 20; earlier(11:50,12:10) = true; earlier(12:10,11:50) = false	minutes=20, true, false
12:00 vs 11:59	minutes = 1; earlier(12:00,11:59) = false	minutes=1, false
9:45 and 13:10 (given program)	585 and 790 min → 205; earlier(arrival, departure) = false	205 false

Growth investigation – 10,000,000 calls, with and without inline, at -O0 and -O2

Method: each iteration builds two Time objects whose values depend on the loop counter and on a volatile variable, so the compiler cannot hoist the call out of the loop; all variants return the same checksum (1205000000), which confirms they compute the same thing. "Inline" = function marked inline; "no inline" = __attribute__((noinline)) forces a real call.

Variant	-O0 inline	-O0 no inline	-O2 inline	-O2 no inline
friend minutesBetween(a, b)	117.0 ms	125.1 ms	20.8 ms	25.1 ms
member a.minutesTo(b)	97.7 ms	125.7 ms	19.3 ms	22.8 ms
public getters (hh(), mm())	167.7 ms	180.1 ms	19.1 ms	69.6 ms

Explanation. Optimisation level matters far more than the inline keyword: going from -O0 to -O2 makes the code about 5–9× faster for every variant (117 → 20.8 ms for the friend). Inline helps a little at -O0 (the call overhead is removed only when the compiler honours the hint) and noticeably when the function is a trivial getter at -O2: the non-inlined getter version costs 69.6 ms against 19.1 ms inlined, because two extra calls per comparison are removed. For the friend and member versions, which are one call each, the gain from inlining at -O2 is about 10–20 %. The compiler is free to ignore the keyword, so inline is a hint, not a guarantee; its other (guaranteed) effect is allowing the definition in a header without a multiple-definition error.

Deployment decision

Keep the **friend functions** for the symmetric operations minutesBetween and earlier: they need the private fields, treat both objects equally, and avoid exposing h and m through getters. Use getters only where outside code genuinely needs the values (for printing). Each friend declaration widens the class's coupling, so keep the number small.

Alternative concept – member functions or getters

Time per call is O(1) in all three designs. Measured (10,000,000 calls): at -O2 friend 20.8 ms, member 19.3 ms, getters 19.1 ms – the same within noise once inlined; at -O0 friend 117 ms, member 98 ms, getters 168 ms – getters are slowest because each comparison makes extra calls. Encapsulation: friend and member functions both access the private data directly without exposing it; public getters expose h and m to every caller, the weakest encapsulation, though they need no friend declaration and give the most flexibility to unrelated code. The member form treats the left object specially (a.minutesTo(b)), the friend form treats both equally.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Chosen program: **Q4 – SavingsAccount mini-statement (shifting array)**, because the bank asks for 50 transactions instead of 10 and the cost of shifting grows with the window size.

A. Diagnose the Limitation (2)

Evidence from my Q4 measurements (10,000,000 deposits): with 10 transactions the shifting version needed 163 ms at -O0 and 85 ms at -O2; with 50 transactions 1192 ms and 134 ms. Moving from 10 to 50 made it 7× slower at -O0 and 1.6× slower at -O2. Cause: record() moves k−1 elements for every transaction once the buffer is full – O(k) work per deposit that is thrown away immediately, because only one slot actually changes.

B. Define the Design Objective (2)

Objective: make the time to record a transaction independent of the window size k (O(1)), so that 10,000,000 deposits with k = 50 take no more than 1.2 × the time with k = 10, at the cost of at most one extra int of memory per account, while the mini-statement output stays identical.

C. Design the Revised Solution (4)

```
class SavingsAccount {
    static const int N = 50;           // window size, one constant
    long accNo; double balance;
    double txn[N];
    int start = 0;                     // index of the oldest entry
    int count = 0;                     // 0..N
public:
    void record(double amt) {
        if (count < N) txn[(start + count++) % N] = amt;
        else { txn[start] = amt; start = (start + 1) % N; }      // overwrite oldest, O(1)
    }
    void miniStatement() const {
        for (int i = 0; i < count; ++i) std::cout << txn[(start + i) % N] << '\n';
    }
};
```

C++ feature used: a private array with a modulo index inside the same class (no library change). Extra memory: one int (start) per account – 4 bytes. New trade-off: indexing logic is a little harder to read, and miniStatement() needs the modulo to print in order; but the public interface (deposit, withdraw, miniStatement) is unchanged, so no calling code changes.

D. Predict Before Implementing (1)

Prediction: the circular version's time per deposit will be about the same for k = 10 and k = 50 (within 10 %), and at k = 50 will be at least 3× faster than shifting at -O2 and 20× faster at -O0. Output of miniStatement() will be byte-identical.

E. Implement and Validate the Redesign (2)

Measure (10,000,000 deposits)	Original (shift)	Redesign (circular)
k = 10, -O0	163 ms	46 ms
k = 50, -O0	1192 ms	49 ms
k = 10, -O2	85 ms	39 ms
k = 50, -O2	134 ms	38 ms
Extra memory per account	–	+4 bytes
miniStatement() output (same 60 deposits)	identical	identical

F. Defend the Final Design (2)

The objective was met: the circular version at k = 50 took 38 ms against 39 ms at k = 10 (ratio 0.97 ≤ 1.2), and 3.5× faster than shifting at -O2 (134/38) and 24× faster at -O0. The prediction held for the trend; the improvement at -O2 for k = 10 (2.2×) was larger than expected for such a short loop. Costs: 4 extra bytes and slightly harder code (modulo arithmetic, which is also a division at -O0). **Where the original may still be preferred:** with k = 10 and a few transactions per second the difference is invisible (a deposit takes nanoseconds), and the shifting code is simpler to read and to verify; a student project or a rarely-called function should keep the simple version. The redesign is justified only when k is large or the operation is called millions of times.

Simulation Screenshot

Compiled with g++ -std=c++17 -Wall -Wextra. The screenshot shows the Q1 prediction program, the call-by-value/reference fix, the six overload selections (Q3), the static-member programs (Q5), the friend-function program with its correctness checks (Q6), and the ambiguous-overload compiler error.

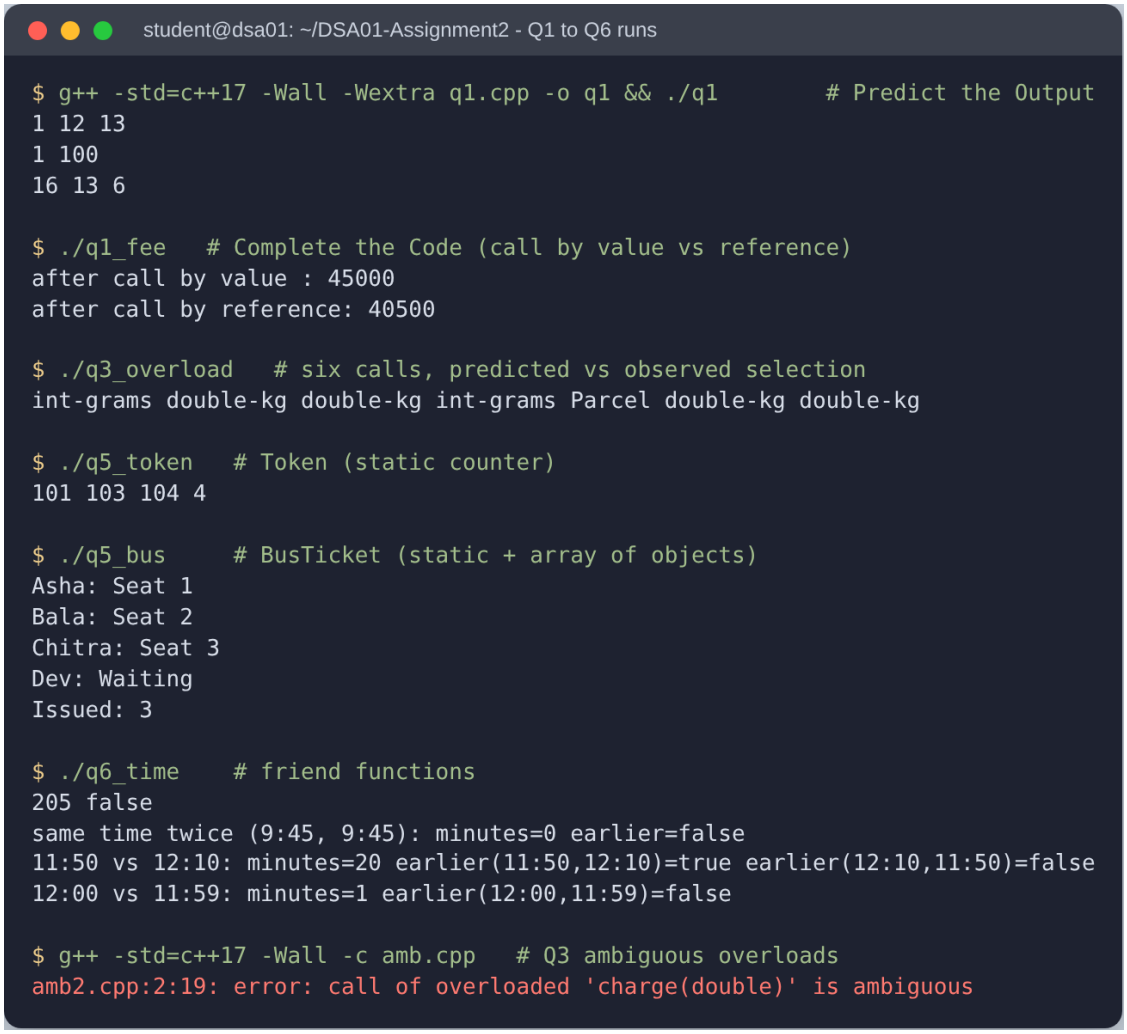


Figure 1 – Terminal run of the Assignment 2 programs.